

JomCode Coding Bootcamp

HTTP, REST and async foundations

Understand, predict, type, run, then practice.

What you will be able to do

- 01 Describe method, path, status and JSON body in an HTTP exchange.
- 02 Choose consistent CRUD endpoints and response codes.
- 03 Explain when Python async helps and when a blocking database call remains synchronous.

Plain meaning → technical view → use

PLAIN MEANING

A browser asks a server to read or change information.

TECHNICAL VIEW

HTTP methods, paths, status codes and JSON form the API contract; async handles waiting.

WHEN YOU USE IT

React sends GET/POST/PATCH/DELETE requests to FastAPI.

TIP

Inspect method, URL, status, and JSON separately in the browser Network tab.

Worked example 1: Request construction



chapter_14_http_rest_and_async.py

```
1 import httpx
2
3 request = httpx.Request("GET", "http://localhost:8000/workshops",
4                          params={"q": "Python", "offset": 0, "limit": 10})
5 print(request.method)
6 print(request.url.path)
7 print(request.url.params["limit"])
```

PREDICT Which request part contains the search text?

Example 1: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
GET  
/workshops  
10
```

WHY THIS HAPPENS

Query parameters represent a catalogue view. This example constructs a request without contacting a server.

Exercise 1: Method and status map

BUILD

Write a mapping for list, create, partial update and successful delete.

CHECK

GET/200, POST/201, PATCH/200, DELETE/204.

IF STUCK

A status describes the outcome, not just the method.

Worked example 2: JSON response



chapter_14_http_rest_and_async.py

```
1 import httpx
2
3 response = httpx.Response(201, json={"id": "w1", "capacity": 20})
4 print(response.status_code)
5 print(response.json()["id"])
```

PREDICT Would you parse JSON after every 204?

Example 2: output and reason

TERMINAL	>_	EXPECTED OUTPUT
----------	----	-----------------

201		
w1		

WHY THIS HAPPENS

No. A successful 204 response has an empty body.

Exercise 2: 204 handling

BUILD

Write `decode_success(response)` returning `None` for 204 and JSON otherwise.

CHECK

204 => `None` without JSON parsing. 200 JSON => object.

IF STUCK

Branch before parsing the body.

Worked example 3: Async waiting



chapter_14_http_rest_and_async.py

```
1 import asyncio
2
3 async def demo():
4     await asyncio.sleep(0)
5     return "Ready"
6
7 print(asyncio.run(demo()))
```

PREDICT Does `async def` automatically make a blocking SQLAlchemy call nonblocking?

Example 3: output and reason

TERMINAL	>_	EXPECTED OUTPUT
----------	----	-----------------

Ready		
-------	--	--

WHY THIS HAPPENS

No. Use a synchronous route for a synchronous database driver, or select a deliberate async driver.

Exercise 3: Origin comparison

BUILD

Use `urlsplit` to compare two local origins. Show why a different port matters.

CHECK

5173 differs from 8000. A path does not change the origin.

IF STUCK

Ignore path while comparing the origin.

Checkpoint

PROVE IT

A learner can explain the request, status and next UI action for one successful and one failed journey.